

Synthetic Data Generation for Trustworthy Data Space Services

Nada Akel

Oszkár Semeráth

nada.akek@edu.bme.hu

semerath.oszkar@vik.bme.hu

Department of Artificial Intelligence and Systems
Engineering, Faculty of Electrical Engineering and
Informatics, Budapest University of Technology and
Economics
Budapest, Hungary

Imre Kocsis

László Gönczy

kocsis.imre@vik.bme.hu

gonczy.laszlo@vik.bme.hu

Department of Artificial Intelligence and Systems
Engineering, Faculty of Electrical Engineering and
Informatics, Budapest University of Technology and
Economics
Budapest, Hungary

Abstract

A data space provides a trusted framework where participants can share and use data under common governance principles and standards. In these ecosystems, data are often processed through components that perform transformations to adapt them to specific use cases. However, such transformations can introduce errors that reduce data quality and lead to inconsistencies between participants; therefore, ensuring their correctness is essential. Testing these components is challenging because they handle complex data structures and validation constraints. In this paper, we propose a data generation approach for data space services that produces a valid and diverse set of test data, compliant with domain constraints. We demonstrate our approach using sample data transformations on data that conform to the Experience API (xAPI) specification, using an evaluation strategy based on mutation testing.

CCS Concepts

• **Information systems** → **Data management systems**; • **Software and its engineering** → *System modeling languages*.

Keywords

Data space, Data generation, Data transformation, Mutation testing

ACM Reference Format:

Nada Akel, Oszkár Semeráth, Imre Kocsis, and László Gönczy. 2026. Synthetic Data Generation for Trustworthy Data Space Services. In *The 41st ACM/SIGAPP Symposium on Applied Computing (SAC '26)*, March 23–27, 2026, Thessaloniki, Greece. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3748522.3779965>

1 Introduction

Context. A data space [7] is a collaborative environment that includes various components that provide, process, and consume data among different partners who may not fully trust each other. Data in such ecosystems may be exchanged using different specifications and formats depending on the domain and use case. One example is the Experience API (xAPI) [13], an e-learning specification for sharing learning experiences, which is used in multiple collaborations,

including the Gaia-X Data Space for Skills and Education [9]. Some components within a data space apply transformations to the data as part of their processing tasks. However, these transformations can introduce consistency errors and faulty data that negatively affect the quality of the shared data, or even insert critical, unrepairable errors into the data space that violate the data invariants. This highlights the need to develop test data generators to verify the transformations used in data space ecosystems.

The broader context of our work is that while the strategic and multifaceted nature of data quality in data ecosystems in general, and data spaces in particular, is increasingly well understood (see, e.g., [1]) as a software tooling and metadata management problem in terms of the data itself, the challenge of guaranteeing "data veracity" in data space *service chains*, where a chain of *data producers* and *consumers* collaborate, has not received much attention yet.

As [1] observes, data ecosystems shift the – formerly "centralized" – responsibility of data quality to the more or less distributed data producers of an ecosystem. Our initial experience in data space development shows that a form of this premise seems to hold for data space service chains, too; for them to deliver the intended value, end-to-end *data transformation quality* depends on each intermediary; with the key challenge (again, similarly to the observations of [1]) that they may not be fully prepared to fulfill this responsibility. For the (even organizationally) distributed data producer/consumer components to be able to *efficiently* coalesce into correct service chains, the already available limited data space development experience already shows that dedicated support is necessary beyond agreeing on simple descriptive assume-guarantee specifications of data format and shape.

Early forms of rich, iteratively refineable specifications of the "data in flight" are already appearing in the data ecosystem context (see, e.g., the Open Data Contract Standard ¹). Our key idea and proposition in this paper is that, on this basis, with the additional introduction of *lightweight formal methods*, we should be able to:

Support early testing in the parallel development of data space service chain components and ease the need for wide case coverage test data sets in service chain "dry runs". In this early phase of development, where real-life data is scarce or unavailable, there is a need for diverse set of synthetic data to test the data requirements and early prototype implementations.



This work is licensed under a Creative Commons Attribution 4.0 International License. *SAC '26, Thessaloniki, Greece*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2294-3/2026/03

<https://doi.org/10.1145/3748522.3779965>

¹See <https://github.com/bitol-io/open-data-contract-standard>, accessed on 2025.10.17.

Support continuous testing of component implementation by detecting unintended changes in data transformations. During operation, synthetically generated data can provide regression tests or aid fault diagnosis by testing the correct behavior of black box data space components.

In this paper, we specifically address the problem of data generation for transformation testing.

Problem. Testing data transformation relies heavily on test data, which imposes several unique challenges. First, testing these components is difficult because of the lack of diversity or coverage metrics that can characterize an effective test suite. Second, while some errors are easily detectable, others are more challenging to identify, as they require a deeper understanding of the transformation logic. Third, existing data are often protected by intellectual property or data protection regulations, making them unavailable in testing scenarios. In summary, a diverse set of focused synthetic data is needed for the efficient testing of data transformation components.

However, it is challenging to provide synthetic data. Existing *test generation approaches* like [32, 17] guided by coverage-based metrics (such as statement or decision coverage) focus on imperative software, making them ineffective in testing data-oriented applications (which may need data-oriented coverage metrics). *Random synthetic data generators* like [22, 26, 34] are constructed to support complex data, which can provide test data with high structural diversity, but they fail to support validation rules that are typically complex structural constraints. Consequently, all randomly generated models will violate at least one rule, and no synthetic test input will get through the initial validation step. *Logic solver-based solutions* like [19, 31] can generate valid models, but their scalability is strictly limited in size.

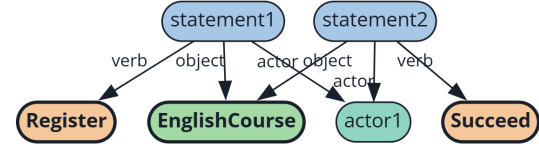
Objectives. The goal of this paper is to introduce a synthetic test data generator based on logic solvers and graph generation [29] to provide effective test data for data space solutions. Moreover, we provide graph-based diversity metrics to support the testing of data space components, thereby increasing trust in these components. Another goal is to evaluate the effectiveness of this approach in revealing errors in data transformation code.

Contribution. We propose a model-driven approach to generate diverse and constraint-aware test data. As a case study, we present the formal representation of the core elements of the xAPI data format using Refinery, a graph generation framework [18]. The triple structure (actor, verb, object), domain profile features, and predefined vocabulary make the xAPI data a good fit to validate the proposed approach. We evaluate the diversity and effectiveness of the generated test data in a mutation testing setup.

Added value. We provide a test data generation technique that supports the development and validation of data space components, automatically creating test inputs for regression testing.

2 Background Overview

In this section, we introduce the mathematical background of our approach. First, subsection 2.1 introduces our case study. Then we introduce logic structures to describe the data format (subsection 2.2), and logic predicates to further requirements on the data (subsection 2.3). Finally, we introduce graph neighborhoods to calculate the diversity of models (subsection 2.4).



(a) Graph Representation

```

1 [{ "id": "statement1",
2   "actor": { "name": "actor1" },
3   "verb": { "id": "register" },
4   "object": { "id": "englishCourse" }
5 }, { "id": "statement2",
6   "actor": { "name": "actor1" },
7   "verb": { "id": "succeed" },
8   "object": { "id": "englishCourse" } }]

```

(b) JSON Representation

Figure 1: Example illustrating the core elements of xAPI data

2.1 Learning record data format

The Experience API (xAPI) standard [13] defines a common and interoperable approach to capture and exchange information about learning activities, whether they occur online or offline. It outlines a format for describing these experiences and how such descriptions can be exchanged electronically. In xAPI, learner interactions are represented as "Statements". A statement is a data structure that records specific experiences or events [13]. Each statement corresponds to an event that occurred at a specific point in time, and when combined, these statements provide a comprehensive account of a learning experience. At its simplest level, an xAPI statement follows the structure "actor-verb-object," representing who performed the action and on what. Additional contextual details, such as results, timestamps, or metadata, can also be included to enrich the description of the experience. Figure 1 illustrates the learning experience of a student (actor1) represented as a sequence of two statements:

- **Statement1:** (actor1, Registers, English course) The student registers for the English course.
- **Statement2:** (actor1, Succeeds, English course) later, the same student succeeds in the course.

The graph representation is shown in Figure 1a, while the corresponding JSON representation is provided in Figure 1b. Together, these illustrate how xAPI statements can represent the content of learning activities in machine-readable form.

2.2 Signatures and Logic Structures

In this paper, we use relational logic structures based on the formalization of [18]. First, the data format is specified over a signature.

Definition 2.1 (Signature). Let $\langle \Sigma, \alpha \rangle$ is a signature, where:

- $\Sigma = s_1, \dots, s_n$ is a finite set of *symbols* defining the set of domain concepts.
- $\alpha : \Sigma \rightarrow \mathbb{N}$ is an *arity function* that denotes the role of each domain concept. Most importantly, domain symbols with

$\alpha(s) = 1$ denote object types, and symbols with $\alpha(s) = 2$ denote relations between objects.

Given a signature, models can be defined by interpretations of the concepts in the signature.

Definition 2.2 (Model). A model is $M = \langle O_M, I_M \rangle$, where:

- O_M denotes the finite number of objects in the model.
- For each symbol $s \in \Sigma$, the interpretation function $I_M(s) : O_M^{\alpha(s)} \rightarrow \{1, 0\}$ assigns a truth-value for each $\alpha(s)$ -long combination of objects in the model, where 1 denotes the true, 0 denotes the false logic value.

Example 2.3. In our example in Figure 1a, the signature of the learning experience includes a binary relation called **verb**, where $\alpha(\text{verb}) = 2$. In the model M , this relation is interpreted with the interpretation function I_M . In this model, object statement1 is connected to **Register** with relation **verb**, which is represented in the logic structure as $I_M(\text{verb})(\text{statement1}, \text{Register}) = 1$.

2.3 Logic Predicates

To impose logic constraints on models, we introduce logic predicates that can be evaluated. We will use predicates to control capture data requirements and control the range of valid test inputs. Logic predicates are constructed using variable symbols, literals (e.g., **actor**(s,a)), equivalence ($=$), logic connectives ($\neg$, $\wedge$, and $\vee$), and quantifiers ($\exists$ and $\forall$). The semantics of the predicates are evaluated in each model following the standard semantic rules [18].

Example 2.4. For example, the predicate

registered(s,a,o) := **actor**(s,a) $\wedge$ **object**(s,o) $\wedge$ **verb**(s,**Register**)

checks whether an actor registered to an object or not, which is defined as the conjunction of three predicates: first, the **actor** of the statement s must be a, then the **object** of the statement s must be o, and finally, the **verb** of the statement s must be **Register**. The predicate **registered**(s,a,o) can be evaluated on the model M presented in Figure 1a with the interpretation function. There, the interpretation of **registered**(statement1,actor1,EnglishCourse) predicate evaluates to 1, and it evaluates to 0 otherwise.

Definition 2.5 (Predicate satisfaction). If a predicate φ is true for all the variables binding in a model M , we say that the model satisfies the predicate, which is denoted by $M \models \varphi$.

If predicates $\varphi_1, \dots, \varphi_n$ denote validation rules, we say that a model M is valid if $M \models \neg\varphi_1, \dots, \neg\varphi_n$, i.e., no validation rule is matching on the model. In our approach, we are using such predicates to express domain constraints.

2.4 Neighborhood-Based Model Equivalence

In order to measure the similarity of two models, we use the neighborhood of a model as proposed by [28, 24]. This metric captures all local properties of objects as neighborhoods up to a given range.

Definition 2.6 (Neighborhood). A neighborhood of a signature $\langle \Sigma, \alpha \rangle$ is defined recursively. For range $r = 0$, the neighborhood is the subset of constant symbols $N_0 = \{c_1, \dots, c_n\}$. For range $r > 0$, the set of neighborhoods are structures $N_r = \Sigma \times \mathbb{N} \times (2^{N_{r-1}})^*$, where for each $\langle s, p, \langle n_1, \dots, n_l \rangle \rangle \in N_r$:

- $s \in \Sigma$ is a symbol,
- p is a position, where $1 \leq p \leq \alpha(s)$, and
- $\langle n_1, \dots, n_{\alpha(s)} \rangle$ is an $\alpha(s)$ -long sequence of neighborhoods, where $n_i \subseteq N_{r-1}$ are neighborhoods for lesser range.

Then, a neighborhood of an object o recursively captures information about all relations in the model where the object is involved.

Definition 2.7 (Neighborhood of an object). Given a range r and a model $M = \langle O_M, I_M \rangle$, neighborhood function $n_r : O_M \rightarrow N_r$ is defined for each object $o \in O_M$ recursively:

- For $r = 0$, if o is a constant c then the $n_0(o) = \emptyset$.
- For $r > 0$, $n_r(o) = \{ \langle s, p, \langle n_{r-1}(o_1), \dots, n_{r-1}(o_m) \rangle \rangle \mid \text{there is a symbol } s \in \Sigma \text{ where } I_M(s)(o_1, \dots, o_m) = 1 \text{ and } o = o_{pos} \}$.

Example 2.8. In Figure 2 three models are shown: M_1 , M_2 and M_3 . Here, in all three cases we would like to calculate the neighborhood of the node object1. First, for range $r = 0$, as per definition, $n_0(\text{object1}) = \emptyset$ for all three models. For range $r = 1$, the neighborhood of the node describes that (1) it is a **Statement** ($\langle \text{Statement}, 1, \langle \emptyset \rangle \rangle$), and it is the target (2nd position of the relation relation, which is denoted $\langle \text{object}, 2, \langle \emptyset, \emptyset \rangle \rangle$). Therefore, the neighborhood of object1 is $\{ \langle \text{Statement}, 1, \langle \emptyset \rangle \rangle, \langle \text{object}, 2, \langle \emptyset, \emptyset \rangle \rangle \}$.

For range $r = 2$ is calculated similarly, but can differentiate the three neighborhoods in the three models. For example:

- In M_1 the object object1 has an incoming reference from a **verb** with **Register**. Therefore, $n_2(\text{object1})$ contains the neighborhood $\langle \text{verb}, 1, \langle \emptyset, \{ \text{Register} \} \rangle \rangle$.
- In M_2 the neighborhood of node object1 for $r = 2$ contains two different kinds of statements: statement2 with the same neighborhood, and statement1 with connection to **Fail** (i.e., $\langle \text{verb}, 1, \langle \emptyset, \{ \text{Fail} \} \rangle \rangle$).
- In M_3 neighborhood of node object1 for $r = 2$ similarly contains two different kinds of statements, but with the **Complete** constant (i.e., $\langle \text{verb}, 1, \langle \emptyset, \{ \text{Complete} \} \rangle \rangle$).

Definition 2.9 (Neighborhood of a Model). Given a range r and a model $M = \langle O_M, I_M \rangle$, the neighborhood of a model is the set of all the neighborhoods in the model: $n_r(M) = \{ n_r(o) \mid \exists o \in O_M \}$.

In summary, the neighborhood of a model recursively captures a vast range of predicates that were true for the objects in the model. In the following, we utilize the neighborhood of a model to characterize or test inputs, or to improve test sets through test prioritization. If the number of neighborhoods is high in a model M (i.e., $|n_r(M)|$ is high), it is more likely to find errors in the data processing code. Similarly, we use the number of neighborhoods as an optimization criteria for test prioritization to improve the chance of finding new errors (i.e., if we have tested the set of neighborhoods T we select the model M_i where $|T \cup n_r(M_i)|$ is maximal).

3 Graph Based Test Data Generation

3.1 Overview of synthetic generation

In this section, we describe the main steps of our proposed approach, as illustrated in Figure 3. The *input* to our approach is a *domain specification*, which defines the data model used to represent the data and *domain constraints*, a set of validations that any generated data must satisfy. These constraints may originate

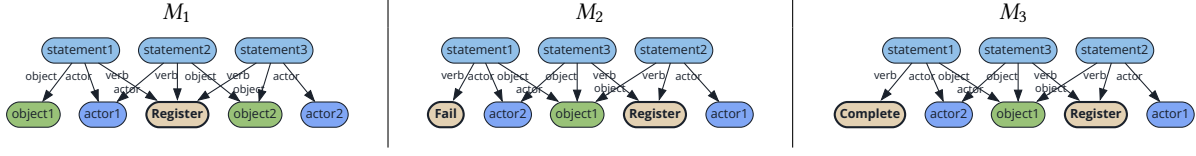


Figure 2: Example of diverse generated models.

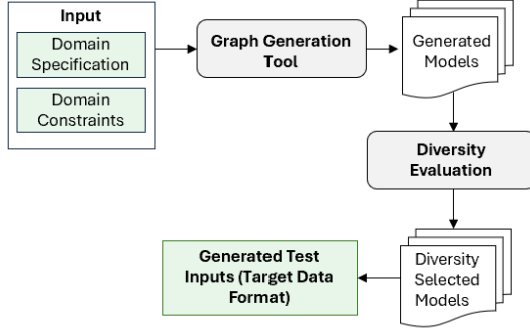


Figure 3: Overview of the proposed approach

from various sources, including those enforced by the data format, domain-specific rules, data quality standards such as ISO/IEC 25000 [14] and ISO 8000 [15], or constraints explicitly specified in data exchange agreements. They may also include constraints introduced by the test engineer to address specific testing scenarios.

Our focus is on formally specified *relational data* with *rich complex structures* that have a generic graph structure. As a running example, we use event trace data captured in the Experience API specification. We propose using a *graph-based generator* [18] to produce test data by following these steps:

First, we begin by building the domain specification in *first-order logic* and formalizing the *core elements* of xAPI data into a *domain model*, as illustrated later, which specifies the structural entities and relationships. Moreover, we define a set of *constraints* that capture additional requirements and domain rules. These constraints can be defined as a set of first-order logic constraints by either the test engineer or the domain developer. (See subsection 3.2.)

Second, the *graph generation* [18] uses the *domain model* and *constraints* to generate a wide range of *candidate models* representing possible instances of xAPI data.

Third, after the generation step, we introduce a *diversity evaluation* phase to measure and *maximize variation* within the generated set. This step enables us to select a *focused subset* of models that capture the *diverse structural* and *semantic* characteristics of the domain, allowing us to detect a similar number of bugs with fewer test data, a process known as test minimization. (See subsection 3.3.)

Finally, the diversity-selected model instances are *serialized* into the *target format* (e.g., JSON), which can serve as test inputs.

In general, our approach enables the *automated generation* of *diverse test data* that aligns with the *domain model* and satisfies the specified validation *constraints*, which can be used to perform manual or regression testing.

3.2 Formal Specification of xAPI Data

In this subsection, we present our formalization of the xAPI data format as a first-order logic theory, capturing the entities of the data format, their relationships, and the constraints.

The xAPI specification for a selected domain is defined using an *xAPI profile*, which is a collection of statement templates and patterns. Each xAPI statement is associated with a *statement template* that describes its usage context and *required data*. The profile also provides a way to refer to *vocabulary concepts* used or referenced in statement templates, such as *verbs* and *activities*.

Our formalization of xAPI data is based on the main elements of the xAPI data format, as illustrated in the example shown in Figure 1. To formally represent the core structure of xAPI data, we model it as logic structures defined over a signature, which provides the vocabulary of the domain, while the model assigns meaning to that vocabulary in terms of concrete objects and their relationships. For the core elements of xAPI statements (statement, actor, verb, and object), the signature is defined as:

$$\Sigma = \{\text{Statement, Actor, Verb, Object, actor, verb, object}\}$$

$$\alpha(\text{Statement}) = \alpha(\text{Actor}) = \alpha(\text{Object}) = 1$$

$$\alpha(\text{actor}) = \alpha(\text{verb}) = \alpha(\text{object}) = 2$$

Additional constant predicates can be defined to restrict the possible values of model elements according to predefined vocabularies, ensuring that the generated data adhere to the allowed vocabulary concepts defined in the profile. For example, verbs and activities may be limited to those defined in the SCORM profile [27] (e.g., **Verb(Fail)** or **Verb(Complete)**). Moreover, specific problem domains may also introduce additional predicates to represent domain-specific constants.

To ensure the validity of the generated data, we formalized a set of constraints that capture essential rules about the data, such as structural properties, multiplicity, and quality requirements related to duplication and contradictions, as defined by data quality standards. These constraints reflect our assumptions about the generated data model and must hold for any valid instance within the domain of application.

Type hierarchy constraints: Ensure that each element in the model is only one of the core types: **Statement**, **Actor**, or **Object**.

Type compliance constraints: Ensure correct relationships between model elements. If a relationship exists between two elements, the source and target must be of the correct types; for example, the source of an **actor** relationship must be a **Statement**, and the target must be an **Actor**.

Multiplicity constraints: Specify the lower and upper multiplicities. They ensure that each **statement** has exactly one **Actor**, one **Verb**, and one **Object**, preventing incomplete or invalid **statements**, such as **statements** with no **Actor** or more than one **Actor**.

Constants constraints: Restrict certain elements, such as **verbs** or activity types, to predefined vocabularies defined in the xAPI profile or domain. For example, **Verbs** could be limited to a controlled set, such as **Fail** or **Complete**.

Next, we are introducing additional validation rules. By incorporating these constraints into our formalization, we ensure that the generated test data not only adheres to the syntactic structure of xAPI but also satisfies additional constraints required by the domain specification or data quality requirements. Let us consider generating a dataset containing information about student activities in a specific academic year, and we have the following domain constraints and requirements. To ensure that the generated data adheres to those requirements, we formalize those constraints using the logic predicate introduced in Section 2.

"No duplicate statements.": The error predicate **duplicationError**(s1, s2) holds when two different **statement** s1 and s2 share the same **actor** a, **verb** v, and **object** o.

$$\text{duplicationError}(s1, s2) := \exists a, o : \text{actor}(s1, a) \wedge \text{actor}(s2, a) \wedge$$

$$\text{object}(s1, o) \wedge \text{object}(s2, o) \wedge \text{verb}(s1, v) \wedge \text{verb}(s2, v) \wedge (s1 \neq s2)$$

This constraint ensures that each unique combination of **actor**, **verb**, and **object** appears only once. Preventing such duplicated **statements** helps maintain the quality of the generated data.

"A student cannot both Pass and Fail the same course.": The error predicate **contradictionError**(s1, s2) identifies contradictory statements related to the same **actor** and **object**. It holds when two **statements** s1 and s2 have the same **actor** student a and the same **object** course o, but their **verbs** show opposite outcomes, one being **Fail** and the other **Succeed**.

$$\text{contradictionError}(s1, s2) := \exists a, o : \text{actor}(s1, a) \wedge \text{actor}(s2, a) \wedge$$

$$\text{object}(s1, o) \wedge \text{object}(s2, o) \wedge \text{verb}(s1, \text{Succeed}) \wedge \text{verb}(s2, \text{Fail})$$

This constraint helps detect inconsistencies in the data, ensuring that the generated or transformed xAPI dataset is consistent.

"A student cannot perform any action on a course unless they are registered in that course.": The error predicate **nonRegistered**(a, o) identifies invalid actions performed by a student who is not registered in a course. It holds when there is a **statement** s where the **actor** student a performs any **verb** v on the **object** course o, but v is not **Register**, and there is no existing registration record for that student and course, which can be detected with negation of pred **registered**(s, a, o).

$$\text{nonRegistered}(a, o) := \exists s, v : \text{actor}(s, a) \wedge \text{object}(s, o) \wedge \text{verb}(s, v) \wedge$$

$$v \neq \text{Register} \wedge \forall x : \neg \text{registered}(x, a, o)$$

$$\text{registered}(s, a, o) := \text{actor}(s, a) \wedge \text{object}(s, o) \wedge \text{verb}(s, \text{Register})$$

This constraint ensures that all actions in the data adhere to valid enrollment relationships, preventing the generation of logically incorrect statements in accordance with the domain constraints.

This formalization maps each core element of xAPI data to a well-defined logic symbol, explicitly encoding the relationships between them. As a result, it supports the automated generation of valid instances, providing a foundation for generating test inputs and validating them. Importantly, Refinery leverages this logic structure to generate models that satisfy the defined constraints, ensuring that all produced xAPI statements conform to the domain specification and capture the intended semantics of the data.

3.3 Evaluation of Test Data Diversity

Our test data generation approach is guided by two factors: we generate valid test data by *ensuring the correctness of the generated tests* and improve the quality of the test suite by *optimizing the diversity of the tests* using the neighborhood representation [28]. First, let us introduce our fault model and fault detection assumption.

Fault Model: Data validation and processing are based on the evaluation of predicates (e.g., filter conditions, group conditions, aggregations, etc.). Instead of a correct predicate φ the developer writes a different predicate φ_f .

Fault Detection: Data processing code processes a model M , validates it, and produces an output. Our fault hypothesis assumes that there exists at least one predicate φ_c that is related to the expected behavior of the program code. However, in the faulty code, the behavior is different, and this faulty behavior can be related to a different predicate φ_f . Then, if $M \models \varphi_c$ and $M \not\models \varphi_f$, or $M \not\models \varphi_c$ and $M \models \varphi_f$, then M can differentiate φ_c and φ_f by differentiating the expected and actual validation result or output.

Therefore, our test generation strategy relies on producing models that can differentiate the most number of predicates. In our approach, we are using neighborhoods to characterize and quantify the number of potential predicates that can activate a failure. Each neighborhood captures a range predicates $\{\varphi_1, \varphi_2, \dots\}$ that is true for that node in the model [24], i.e., if $X \in n(M)$ then $M \models \varphi_1, \varphi_2, \dots$ is true as well. Therefore, if a model M contains that neighborhood ($X \in n(M)$), and the fault consists of a predicate φ_c replaced by another faulty predicate φ_f , it has the potential to activate a failure if there is a φ_i code where $\varphi_i \implies \varphi_c$ and $\varphi_i \implies \neg \varphi_f$, or $\varphi_i \implies \neg \varphi_c$ and $\varphi_i \implies \varphi_f$.

Our test optimization strategy relies on selecting test data with the most number of different neighborhoods to optimize its effectiveness (for some r). For this purpose, we use the hill climbing algorithm [30] to optimize the number of different neighborhoods in our selected tests. For the optimization, we have a set of test data candidates $C = \{M_1, \dots, M_n\}$, and we would like to select a list of priority inputs given size limit $P_1, P_2, \dots, P_{limit}$.

- (1) First, we select the model with the most number of different neighborhoods, where $|n_r(M_i)|$ is maximal. We include this model to the priority list $P_1 = M_i$.
- (2) If do not have enough tests, then we summarize the number of neighborhoods in our priority list $PN = \bigcup_{P \in P_1, \dots, P_m} n_r(P)$, and select the model M_i that is not selected before, and it has the most number of new neighborhoods, where $|n_r(M_i) \setminus PN|$ is maximal. We include M_i to the priority list $P_{m+1} = M_i$.
- (3) If we have enough models in the priority list, we terminate with the list $P \in P_1, \dots, P_{limit}$.

4 Evaluation

To evaluate our approach, we performed preliminary measurements to address the following research questions.

- **RQ1:** How do the number and size of generated models impact the test generation time?
- **RQ2:** How effective are our generated test data in finding errors in code that performs data transformations?
- **RQ3:** How effective is our diversity metric in minimizing the size of the generated test suite?

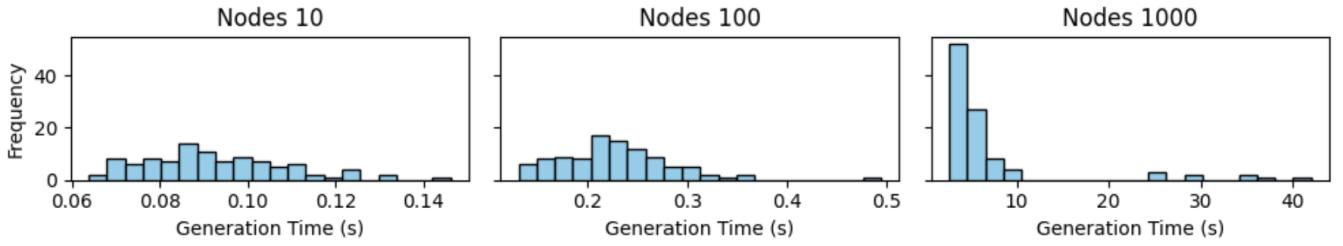


Figure 4: Model generation time per model size.

4.1 RQ1: measurement setup

To answer RQ1, we conducted model generation experiments aligned with our xAPI case study. First, we utilized the metamodel in Refinery to describe a simplified xAPI data format that conforms to the model used in the case study. Additionally, we formalized the constraints derived from the data requirement as graph predicates on this model. Second, we defined three model sizes: one with 10 nodes, another with 100 nodes, and the largest with 1000 nodes. Last, we generate a large set of test input models using the Refinery graph generator.

We generate 100 models for each model size independently. Moreover, for each model size, we performed 10 warm-up iterations that were not included in the measurements. Between measured runs, we invoked the garbage collector and introduced a 0.5-second sleep period to ensure that each model generation did not overlap with the previous one. We measured the time in nanoseconds.

4.2 RQ1: analysis of the results

The problem preprocessing time was approximately 4.57 seconds, which is needed only once when the generator starts, and it is independent of the number of models to be generated. Then the measurement time is dominated by the generation of valid graph models. The model generation times for the three different model sizes are shown in Figure 4. The translation of the graph model to JSON was negligible (less than 0.5 seconds).

The histograms show that for our case study, the generation time increases with model size. For small models (10 nodes), the generation is high-speed and consistent, with most runs taking less than 0.12 seconds. For medium models (100 nodes), times are slightly higher, ranging up to 0.5 seconds. However, for large models (1,000 nodes), the generation time ranges from a few seconds to more than 40 seconds.

RQ1: The size of the generated models directly affects the performance of test data generation. Larger models require longer generation times, while smaller models are generated almost instantly. The runtime for the largest model is still acceptable for test generation.

4.3 RQ2 & RQ3: measurement setup

To evaluate the effectiveness of the generated test suite in detecting faults within the transformation code, we adopted a mutation testing setup, as illustrated in Figure 5. The process contains the following steps:

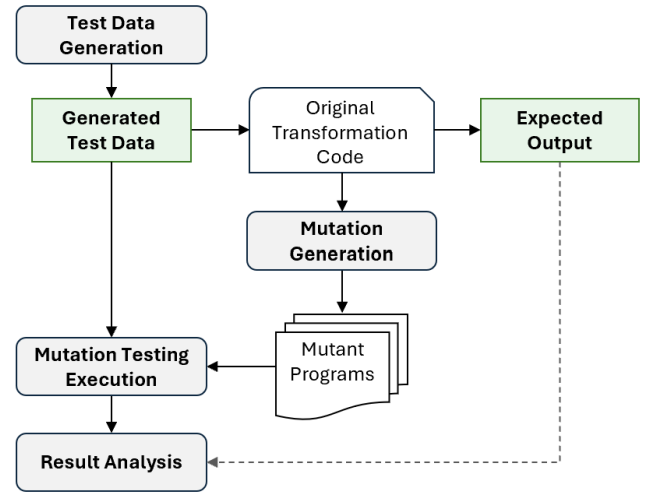


Figure 5: Mutation testing setup

- **Test data generation:** We used the same setup as in the previous section, generating 100 models for each of the three model sizes described earlier.
- **Reference output:** We executed a Python transformation code that performs aggregation, sorting, and filtering on the data. The resulting outputs were hashed and stored to serve as the reference results of the transformation.
- **Mutation generation:** We applied mutmut [10], a Python mutation testing library, to introduce mutants into the transformation code.
- **Mutation testing:** Each mutant version of the code was executed using the generated test data, and the produced outputs were compared with the reference results. If a test case detected a behavioral difference and caused the mutant to fail, the mutant was considered killed; otherwise, the test data could not detect that fault.

To address RQ3, after running the mutation testing using the original model generation order, we reordered the test data using an optimized order based on their previously defined neighborhood values, which prioritize the test data according to their diversity, and then rerun this step.

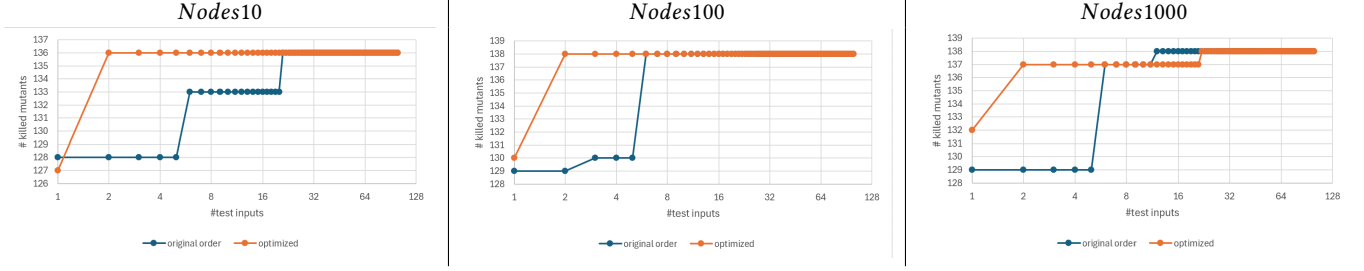


Figure 6: Mutant killing results for two model ordering sequences (log scale).

4.4 RQ2: analysis of the results

We evaluated the effectiveness of the generated test data using the number of mutants killed. Three test suites with different models sizes were used, and for each suite, test data were gradually added to measure the cumulative percentage of killed mutants.

A total of 147 mutants were produced from the transformation code using mutmut. Among them, approximately 6.8% were particularly difficult to kill, depending on the model size, and around 6.0% were identified as unkillable (e.g., the mutation has not introduced any error). The remaining mutants were detectable by any of the generated test data. The following table summarizes the percentage of killed mutants per suite.

	Nodes10	Nodes100	Nodes1000
Killed mutants(%)	92.5%	93.9%	93.9%

The results show that all three test suites achieved comparable effectiveness, killing approximately 93% of the generated mutants, with minor variation across model sizes. In our evaluation, the duration of test execution time was negligible (averaging approximately 0.24 milliseconds per test data, as reported by mutmut) compared to the model generation time.

Figure 6 shows the mutant-killing results for two test data orders (blue for the original order and orange for the optimized order) on a logarithmic scale.

RQ2: The results show that the generated test data effectively detects faults in the transformation code regardless of model size. Confirming that the proposed approach produces valid and effective test data for detecting transformation errors.

4.5 RQ3: analysis of the results

Figure 7 shows how the neighborhood values grow as more test data are added. For the 10-node model, diversity reaches its maximum after 20 generated models, meaning that adding more tests does not increase diversity. For the 100-node and 1000-node models, the neighborhood values continue to increase more slowly, meaning that larger models produce more diverse data and require more test inputs to achieve their full diversity.

Figure 6 compares the mutant-killing performance of the original and optimized test orders for models of 10, 100, and 1000 nodes. The optimized order (orange line) consistently achieves higher killing rates with fewer test inputs than the original order (blue line).

For the 10- and 100-node models, the optimized order reaches full mutant coverage earlier, requiring only two test inputs to kill all mutants. For the 1000-node model, it was able to kill most of

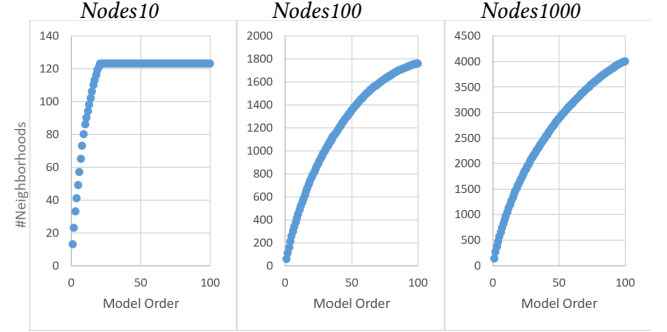


Figure 7: Test data diversity.

the mutants using only two test inputs, while the remaining one mutant required adding around ten extra test inputs compared to the original order.

RQ3: By using the proposed diversity metrics as heuristics for test prioritization, the same coverage can be achieved with fewer input data compared to the default model sequence produced by the model generator.

5 Related Work

The concept of data spaces imposes and requires new verification and validation (V&V) solutions [4, 12], as independent stakeholders placed in complex and changing organization contribute to the shared data; *data quality* is a central question in all V&V solutions.

Some solutions focus on improving data quality by *ensuring rigorous consensus protocols*. One paper proposes the use of blockchain technologies [8], introducing fine-grained access control and identity management [20]. Other approaches focus on the merging of data using description logic and ontologies [6], which proposes the robust merging of data formats.

Another group of approaches focuses on improving *data sharing protocol* between industries [11] and organizations [16]. Some case study focuses on the precise exploration of data flow between organizations [25], or to formulate the enabled or legal data sharing protocols [5, 33].

Our approach focuses on the *precise formalization of the data format* and the testing of data processing and transformation steps used in data spaces. There are some techniques that focus on semantic testing, such as Ontology with Semantic Web Rule Language (SWRL), Cognipy with Controlled Natural Language (CNL), and

Shapes Constraint Language (SHACL). Rabelo et al. [23] conducted a comparative study of four xAPI validation tools and found SmartLAK, an ontology-based approach, to be the most effective. Vidal et al. [6] proposed an ontology for xAPI data using OWL and SHACL to define constraints and validate data. Their ontology captures the semantics of the xAPI specification with nearly 150 axioms and integrity constraints to ensure correctness. Pareti et al. [21] reviewed SHACL as a constraint language, explaining its basic concepts, constructs, components, and interactions for validating RDF graphs. Finally, there are some approaches aiming for the direct generation of JSON files [2, 3]. While those approaches are able to support json format, they do not support general first-order logic (e.g., [3] misses negation), and they are not evaluated in testing.

Our approach uses first-order logic predicates to capture the data format. An immediate benefit of our approach is the ability to efficiently instantiate [29] a wide range of diverse test data [28].

6 Conclusion

We introduced a model-driven approach for generating diverse test data to support the validation of data transformation components in data space ecosystems. By combining logic-based graph generation with diversity-based prioritization, the approach generates test inputs that are syntactically correct, meet basic validation constraints, and comply with domain-specific requirements. We have formalized the core elements of the xAPI statements through the use of predicates and constraints. Our evaluation of the xAPI model demonstrated that synthetic data can efficiently detect faults in transformation code, even with smaller model sizes, and that prioritizing test inputs using diversity further improves the results.

As future work, we plan to enhance our approach to automatically recognize the data model from the data schema. We will evaluate how the technique can be used in different phases of data space development and operation, with a focus on providing test data for complex service chains.

Acknowledgments. This work has been partially supported by the European Union in the frame of the project European Dataspace for Growth and Education – Skills, short: EDGE Skills (101123471), and the Department of Navy award (N629092412063) issued by the Office of Naval Research.

References

- [1] Marcel Altendeitering, Tobias Moritz Guggenberger, and Frederik Möller. A Design Theory for Data Quality Tools in Data Ecosystems: Findings from Three Industry Cases. *Data & Knowledge Engineering*, 153:102333, 2024.
- [2] Lyes Attouche, Mohamed-Amine Baazizi, and Dario Colazzo. *Optimistic Data Generation for JSON Schema*, pages 119–152. Springer Berlin Heidelberg, Berlin, Heidelberg, 2024.
- [3] Lyes Attouche, Mohamed-Amine Baazizi, Dario Colazzo, Giorgio Ghelli, Carlo Sartiani, and Stefanie Scherzinger. Witness Generation for JSON Schema. *Proceedings of the VLDB Endowment*, 15(13):4002–4014, 2022.
- [4] Claudia P Ayala, Besim Bilalli, Cristina Gómez, Jose-Norberto Mazón, Oscar Romero, et al. Challenges to Enforce Data Quality in Data Spaces. 2025.
- [5] Cristina-Georgiana Calancea. A Review on Business-to-Business Data Sharing Technical Solutions. *International Journal on Optimization and Applications*, 2(01), 2022.
- [6] Qiushi Cao, Franco Giustozzi, Cecilia Zanni-Merk, François de Bertrand de Beuvron, and Christoph Reich. Smart Condition Monitoring for Industry 4.0 Manufacturing Processes: An Ontology-Based Approach. *Cybernetics and Systems*, 50(2):82–96, 2019.
- [7] CEN-CENELEC Workshop Agreement. CWA 18125:2024 - Trusted Data Transaction. Technical report, CEN-CENELEC, 2024. Accessed: 2025-01-21.
- [8] Andrea Roberta Costagliola, Carlo Mazzocca, Armir Bujari, Rebecca Montanari, and Paolo Bellavista. VESPACE: A Verifiable Blockchain-Based Data Space Solution to Empower the Data Economy. *Computer Communications*, 239:108180, 2025.
- [9] Gaia-X. Gaia-X Data Space for Skills and Education (DaSES). Accessed: 2025-10-07.
- [10] Milos Gligoric, Vilas Jagannath, and Darko Marinov. MuTMuT: Efficient Exploration for Mutation Testing of Multithreaded Code. In *2010 third international conference on software testing, verification and validation*, pages 55–64. IEEE, 2010.
- [11] Laurence Goasdouff, 2021. Data Sharing Is a Business Necessity to Accelerate Digital Business.
- [12] Philipp Hagenhoff, Steffen Biehs, Frederik Möller, and Boris Otto. Designing a Reference Architecture for Collaborative Condition Monitoring Data Spaces: Design Requirements and Views. In Munir Mandviwalla, Matthias Söllner, and Tuure Tuunanen, editors, *Design Science Research for a Resilient Future*, pages 355–369, Cham, 2024. Springer Nature Switzerland.
- [13] IEEE. 9274.1.1 xAPI Base Standard Overview. Accessed: 2025-10-07.
- [14] International Organization for Standardization. ISO/IEC 25000:2014: Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Guide to SQuaRE, 2014.
- [15] International Organization for Standardization. ISO 8000-61:2016: Data quality – part 61: Data quality management: Process reference model, 2016.
- [16] Ilka Jussen, Frederik Möller, Julia Schweihoff, Anna Gieß, Giulia Giusani, and Boris Otto. Issues in Inter-Organizational Data Sharing: Findings From Practice and Research Challenges. *Data & Knowledge Engineering*, 150:102280, 2024.
- [17] Stephan Lukaszczuk and Gordon Fraser. Pynguin: Automated Unit Test Generation for Python. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings, ICSE '22*, page 168–172, New York, NY, USA, 2022. Association for Computing Machinery.
- [18] Kristóf Marussy, Attila Ficsor, Oszkár Semeráth, and Dániel Varró. Refinery: Graph Solver as a Service: Refinement-based Generation and Analysis of Consistent Models. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, pages 64–68, 2024.
- [19] Aleksandar Milicevic, Joseph P Near, Eunsuk Kang, and Daniel Jackson. Alloy*: A General-Purpose Higher-Order Relational Constraint Solver. *Formal Methods in System Design*, 55(1):1–32, 2019.
- [20] Alexander Mühle, Andreas Grüner, Tatiana Gayvoronskaya, and Christoph Meinel. A Survey on Essential Components of a Self-Sovereign Identity. *Computer Science Review*, 30:80–86, 2018.
- [21] Paolo Pareti and George Konstantinidis. A Review of SHACL : from Data Validation to Schema Reasoning for RDF Graphs. *Reasoning Web International Summer School*, pages 115–144, 2021.
- [22] Saheed Popoola, Dimitrios S Kolovos, and Horacio Hoyos Rodriguez. EMG: A Domain-Specific Transformation Language for Synthetic Model Generation. In *International Conference on Theory and Practice of Model Transformations*, pages 36–51. Springer, 2016.
- [23] Thomas Rabelo, Manuel Lama, Juan C Vidal, and Ricardo Amorim. Comparative Study of xAPI Validation Tools. In *2017 IEEE Frontiers in Education Conference (FIE)*, pages 1–5. IEEE, 2017.
- [24] Arend Rensink and Dino Distefano. Abstract Graph Transformation. *Electronic Notes in Theoretical Computer Science*, 157(1):39–59, 2006.
- [25] Saonee Sarker and Suprateek Sarker. Exploring Agility in Distributed Information Systems Development Teams: An Interpretive Study in an Offshoring Context. *Information systems research*, 20(3):440–461, 2009.
- [26] Markus Scheidgen. Generation of Large Random Models for Benchmarking. *BigMDE@ STAF*, 1406:1–10, 2015.
- [27] SCORM. SCORM Profile xAPI. <https://adl.gitbooks.io/scorm-profile-xapi/content/>, 09/09/2021. Accessed: 2025-10-07.
- [28] Oszkár Semeráth, Rebeka Farkas, Gábor Bergmann, and Dániel Varró. Diversity of Graph Models and Graph Generators in Mutation Testing. *International Journal on Software Tools for Technology Transfer*, 22(1):57–78, 2019.
- [29] Oszkár Semeráth, András Szabolcs Nagy, and Dániel Varró. A Graph Solver for the Automated Generation of Consistent Domain-Specific Models. In *Proceedings of the 40th international conference on software engineering*, pages 969–980, 2018.
- [30] Russell Stuart and Norvig Peter. Artificial Intelligence a Modern Approach Third Edition, 2010.
- [31] Emina Torlak and Daniel Jackson. Kodkod: A Relational Model Finder. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 632–647. Springer, 2007.
- [32] Heila Van Der Merwe, Brink Van Der Merwe, and Willem Visser. Verifying Android Applications Using Java Pathfinder. *ACM SIGSOFT Software Engineering Notes*, 37(6):1–5, 2012.
- [33] Monika Woźniak-Cichuta. Business-to-Business Data Sharing within the EU Digital Market (2019-2021). Available at SSRN 4486095, 2023.
- [34] José Antonio Zamudio Amaya, Marius Smytzek, and Andreas Zeller. FANDANGO: Evolving Language-Based Testing. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):894–916, 2025.